	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Aldo Hernandez Fuentes

Asignatura: Modelos De Programacion Orientada a Objetos

Grupo: 7

No de Práctica(s): Practica NO. 7

Integrante(s): Torres Sanchez Efren

No. de lista o brigada: 13

Semestre: 2027/01

Fecha de entrega: 18/10/2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:
Implementar los conceptos de herencia en un lenguaje de programación orientado a objetos.

Ejercicio 1:

Descripcion del problema: El problema consiste en desarrollar un programa en Java que aplique la herencia simple. Para ello, se debe crear una clase Animal que reúna las características y comportamientos comunes a cualquier animal, y una clase Perro que herede sus atributos y métodos. Además de lo heredado, Perro debe agregar información y acciones propias, y redefinir aquellos comportamientos que en un perro funcionan de forma distinta.

Diseño de la solucion:
Para este ejercicio se crearon dos clases relacionadas por herencia, además de una clase de prueba. La clase Animal es la superclase y representa a un animal en general. Contiene tres atributos: nombre, edad y especie, declarados como protected. Esto significa que permanecen ocultos para las clases externas, pero las subclases pueden utilizarlos directamente. Su constructor recibe estos tres datos para inicializar el objeto, y cuenta con los métodos comer(), dormir() y hacerSonido(), que devuelven un texto describiendo la acción del animal. También sobrescribe el método toString(), heredado de la clase Object, para devolver la información del animal en una sola cadena.

La clase Perro es la subclase y se declara con la palabra extends Animal, por lo que hereda todos los atributos y métodos de Animal sin necesidad de volver a escribirlos. Agrega el atributo propio raza y el método propio traerPelota(), que solo tiene sentido para un perro. Su constructor recibe el nombre, la edad y la raza, y reutiliza el constructor de Animal mediante super(), enviando siempre "Canino" como especie. Además, sobrescribe el método hacerSonido() para que el perro ladre, y el método toString() para agregar la raza a la información que ya genera Animal.

Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio1, donde cada clase se encuentra en su propio archivo: Animal.java, Perro.java y PruebaAnimal.java. Al compilar con javac se genera un archivo .class por cada clase. La clase PruebaAnimal contiene el método main y funciona como punto de entrada del programa, mientras que Animal y Perro únicamente definen el modelo. Separar las clases de esta forma permite que cada

archivo tenga una sola responsabilidad y que el modelo pueda reutilizarse en otros programas.

Fragmentos relevantes del código:

- Atributos protegidos de la superclase: Este fragmento muestra la declaración de los atributos de Animal con el modificador protected y el constructor que los inicializa. A diferencia de private, protected permite que Perro acceda a nombre directamente en sus propios métodos, como en:

traerPelota().

```
public class Animal {
   protected String nombre;
   protected int edad;
   protected String especie;

   public Animal(String nombre, int edad, String especie) {
      this.nombre = nombre;
      this.edad = edad;
      this.especie = especie;
   }
}
```

Herencia y reutilización del constructor: La palabra extends establece que Perro hereda de Animal. En su constructor, la llamada a super() debe ser la primera instrucción: ejecuta el constructor de Animal para inicializar los atributos heredados. Después se inicializa el único atributo propio, raza.

```
public class Perro extends Animal {
   private String raza;

   public Perro(String nombre, int edad, String raza) {
      super(nombre, edad, "Canino");
      this.raza = raza;
   }
}
```

Sobrescritura de métodos: La anotación @Override indica que Perro redefine un método heredado. En hacerSonido() se reemplaza por completo el comportamiento de Animal, mientras que en toString() se reutiliza el resultado del padre con super.toString() y solo se le agrega la raza.

```
@Override
public String hacerSonido() {
   return nombre + " dice: ¡Guau guau!";
}

@Override
public String toString() {
   return "Perro: " + super.toString()
      + " - Raza: " + raza;
}
```

Prueba con un arreglo de la superclase: En PruebaAnimal se declara un arreglo de tipo Animal que guarda tanto un Animal como un Perro. Al recorrerlo, cada objeto responde con su propia versión de los métodos. Como traerPelota() solo existe en Perro, se utiliza instanceof para verificar el tipo real del objeto y un cast para poder llamarlo.

```
Animal[] animales = new Animal[2];
animales[0] = new Animal("Nemo", 2, "Pez");
animales[1] = new Perro("Firulais", 4, "Labrador");

for (Animal a : animales) {
   System.out.println(a);
   System.out.println("  " + a.hacerSonido());
   if (a instanceof Perro) {
      Perro p = (Perro) a;
      System.out.println("  " + p.traerPelota());
   }
}
```

Funcionamiento general del programa:

Al ejecutar el programa, Java comienza desde el método main de la clase PruebaAnimal. Se crea un arreglo de tipo Animal con dos elementos: un objeto Animal que representa a un pez llamado Nemo y un objeto Perro llamado Firulais, de raza Labrador. Después, un ciclo for recorre el arreglo e imprime cada objeto, lo que llama automáticamente a su método toString(). Para cada uno se llaman comer() y dormir(), que Perro utiliza tal como los heredó, y hacerSonido(), que en el pez muestra un sonido genérico y en el perro muestra un ladrido gracias a la sobrescritura. Finalmente, al detectar mediante instanceof que el segundo objeto es un Perro, se llama a traerPelota(), el método exclusivo de la subclase.

Pruebas y Resultados:

```
Nemo (Pez, 2 años)
  Nemo está comiendo.
  Nemo está durmiendo.
  Nemo hace un sonido.
Perro: Firulais (Canino, 4 años) - Raza:
Labrador
  Firulais está comiendo.
  Firulais está durmiendo.
  Firulais dice: ¡Guau guau!
  Firulais trae la pelota.
```

Ejercicio 2:

Descripcion del problema: El problema consiste en diseñar en Java una jerarquía de clases para representar figuras geométricas. Se debe crear una clase base Figura con las características que comparten todas las figuras, y a partir de ella las clases Circulo y Rectangulo, cada una con sus atributos propios. Aunque todas las figuras pueden calcular su área y su perímetro, cada una lo hace con una fórmula distinta, por lo que las subclases deben redefinir esos cálculos.

Diseño de la solucion:
Para este ejercicio se crearon tres clases que forman una jerarquía, además de una clase de prueba. La clase Figura es la superclase y contiene los atributos nombre y color, declarados como protected, ya que son datos que tiene cualquier figura. Su constructor recibe ambos valores para inicializar el objeto. También define los métodos calcularArea() y calcularPerimetro(), que en una figura genérica devuelven 0, porque sin medidas no es posible calcularlos; su propósito es que cada subclase los redefina. Por último, sobrescribe toString() para devolver el nombre y el color de la figura.

La clase Circulo hereda de Figura y agrega el atributo propio radio. Su constructor recibe el color y el radio, y envía a Figura el nombre "Círculo" mediante super(). Sobrescribe calcularArea() con la fórmula $\pi \cdot r^2$, calcularPerimetro() con $2 \cdot \pi \cdot r$ y toString() para agregar el radio.

La clase Rectangulo también hereda de Figura y agrega los atributos propios base y altura. Sobrescribe los cálculos con base·altura para el área y 2·(base + altura) para el

perímetro. Además, incluye el método propio esCuadrado(), que devuelve true cuando la base y la altura son iguales.

Circulo y Rectangulo son clases hermanas: ambas son una Figura, pero no tienen ninguna relación directa entre sí. Cada una conserva lo común que hereda de Figura y define solamente lo que la hace diferente.

Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio2, con un archivo por clase: Figura.java, Circulo.java, Rectangulo.java y PruebaFiguras.java. Al compilar se genera un archivo .class por cada una. La clase PruebaFiguras contiene el método main y se encarga de crear las figuras y mostrar sus resultados, mientras que las otras tres clases forman el modelo de la jerarquía.

- **Método pensado para sobrescribirse:** En Figura, calcularArea() devuelve 0. Este método no se usa directamente; existe para que todas las figuras tengan la misma forma de pedir su área, aunque cada subclase la calcule a su manera**.**

```
public class Figura {
   protected String nombre;
   protected String color;

   public double calcularArea() {
      return 0.0;
   }
}
```

- **Atributos propios y constructor de la subclase:** Rectangulo declara base y altura, que Figura no tiene. Su constructor envía el nombre y el color a la superclase con super() y después inicializa sus propios atributos. También agrega el método esCuadrado(), exclusivo de esta clase.

```
public class Rectangulo extends Figura {
   private double base;
   private double altura;

   public Rectangulo(String color, double base,
         double altura) {
      super("Rectángulo", color);
      this.base = base;
```

```
        this.altura = altura;
    }

    public boolean esCuadrado() {
        return base == altura;
    }
}
```

- **Sobrescritura de los cálculos:** Cada subclase redefine calcularArea() con su propia fórmula. En toString() se reutiliza el texto de Figura con super.toString() y solo se agregan las medidas.

```
@Override
public double calcularArea() {
    return Math.PI * radio * radio;
}

@Override
public String toString() {
    return super.toString()
        + " - Radio: " + radio;
}
```

Recorrido de la jerarquía: En PruebaFiguras, un arreglo de tipo Figura guarda objetos de ambas subclases. Al llamar a calcularArea(), Java ejecuta la versión del objeto real. Como esCuadrado() solo existe en Rectangulo, se verifica el tipo con instanceof antes de usarlo.

```
for (Figura f : figuras) {
    System.out.println(f);
    if (f instanceof Rectangulo) {
        Rectangulo r = (Rectangulo) f;
        System.out.println("  ¿Es cuadrado? "
            + (r.esCuadrado() ? "Sí" : "No"));
    }
    double area = f.calcularArea();
    areaTotal += area;
}
```

Funcionamiento general del programa:

Al ejecutar el programa, el método main de PruebaFiguras crea un arreglo de tipo Figura con tres elementos: un círculo rojo de radio 3, un rectángulo azul de 4 × 6 y un rectángulo verde de 5 × 5. Después, un ciclo for recorre el arreglo e imprime cada figura mediante su toString(). Para los rectángulos, el programa verifica con instanceof si se trata de un cuadrado, y detecta que el verde sí lo es. Luego se calculan el área y el perímetro de cada figura, cada una con su propia fórmula, y las áreas se van sumando. Al final se muestra el área total de las tres figuras, 77.27.

Pruebas y Resultados:

```
Círculo Rojo - Radio: 3.0
  Área: 28.27  Perímetro: 18.85
Rectángulo Azul - 4.0 x 6.0
  ¿Es cuadrado? No
  Área: 24.00  Perímetro: 20.00
Rectángulo Verde - 5.0 x 5.0
  ¿Es cuadrado? Sí
  Área: 25.00  Perímetro: 20.00
Área total: 77.27
```

Ejercicio 3:

Descripcion del problema: El problema consiste en modelar en Java una empresa que cuenta con varias categorías de empleados: gerentes, desarrolladores y becarios. Todos comparten datos básicos, como su número de empleado, su nombre y su salario base, pero cada categoría calcula su salario de manera diferente y se presenta con información particular. Se busca reutilizar el constructor de la clase base en cada categoría y sobrescribir los métodos de presentación, para calcular al final la nómina total de la empresa.

Diseño de la solucion:

Para este último problema se utilizaron cuatro clases del modelo y una clase de prueba. La clase Empleado es la superclase y representa a cualquier trabajador de la empresa. Contiene los atributos numEmpleado, nombre y salarioBase, declarados como protected para que las categorías puedan usarlos. Su constructor recibe estos tres datos, y cuenta con el método calcularSalario(), que por defecto devuelve el salario base, y con dos métodos de presentación: presentarse(), que devuelve un saludo del empleado, y toString(), que devuelve su nombre y número.

A partir de Empleado se definen tres categorías, cada una con sus atributos propios y su propia forma de calcular el salario. Las tres subclases reutilizan el constructor de Empleado mediante super(numEmpleado, nombre, salarioBase) y solo inicializan sus atributos propios. Además, sobrescriben calcularSalario(), presentarse() y toString(). En los tres casos, el método sobrescrito llama primero a la versión del padre con super y después agrega lo que corresponde a su categoría. La clase Gerente incluye también el método getDepartamento(), que solo existe en ella. Gerente, Desarrollador y Becario son un Empleado, por lo que todos pueden guardarse en un mismo arreglo de tipo Empleado y tratarse de la misma forma al calcular la nómina.

Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio3, con un archivo por clase: Empleado.java, Gerente.java, Desarrollador.java, Becario.java y PruebaEmpresa.java. Al compilar se genera un archivo .class por cada una. La clase PruebaEmpresa contiene el método main, donde se arma la plantilla de la empresa y se ejecutan las operaciones, mientras que las otras cuatro clases definen el modelo.

Fragmentos relevantes del código:

Reutilización del constructor: El constructor de Gerente recibe cinco datos. Los tres que son comunes a cualquier empleado se envían a Empleado mediante super(), y en Gerente solo se asignan el departamento y el bono. De esta manera, el código de inicialización no se repite en cada categoría.

```
public Gerente(String numEmpleado, String nombre,
        double salarioBase, String departamento,
        double bono) {
    super(numEmpleado, nombre, salarioBase);
    this.departamento = departamento;
    this.bono = bono;
}
```

Cálculo del salario a partir del padre: Cada categoría sobrescribe calcularSalario(), pero en lugar de usar el salario base directamente, obtiene primero el resultado de Empleado con super.calcularSalario() y después le aplica su propia regla. En Desarrollador, se suma el pago de las horas extra.

```
@Override
public double calcularSalario() {
   double salario = super.calcularSalario();
   salario = salario + horasExtra * 250.0; // pago por hora extra
   return salario;
}
```

Sobrescritura de los métodos de presentación: presentarse() de Empleado devuelve un saludo general. Cada subclase lo sobrescribe, reutiliza ese saludo con super.presentarse() y le agrega una frase propia de su categoría. Lo mismo ocurre con toString(), que antepone el puesto al nombre del empleado.

```
@Override
public String presentarse() {
   return super.presentarse()
      + " Dirijo el área de " + departamento + ".";
}

@Override
public String toString() {
   return "Gerente: " + super.toString();
}
```

Plantilla y nómina de la empresa: En PruebaEmpresa, el arreglo plantilla es de tipo Empleado, pero guarda objetos de las tres categorías. Al recorrerlo, cada objeto ejecuta su propia versión de presentarse() y calcularSalario(), y los salarios se acumulan en la nómina. Para el gerente se usa instanceof y un cast con el fin de consultar su departamento.

```
for (Empleado e : plantilla) {
   System.out.println(e);
   System.out.println("  " + e.presentarse());
   if (e instanceof Gerente) {
      Gerente g = (Gerente) e;
      System.out.println("  -> Responsable del departamento "
```

```
            + g.getDepartamento() + ".");
    }
    double salario = e.calcularSalario();
    nomina += salario;
}
```

Funcionamiento general del problema: Al ejecutar el programa, el método main de PruebaEmpresa crea un arreglo de tipo Empleado con cuatro integrantes: un gerente del área de Tecnología, dos desarrolladores, uno de Java y otro de Python, y un becario de la UNAM. Cada objeto se crea con el constructor de su categoría, que a su vez llama al de Empleado. Después, un ciclo for recorre la plantilla: cada empleado se muestra con su puesto mediante toString() y se presenta con su propia versión de presentarse(). Al detectar con instanceof que el primer empleado es un Gerente, se muestra el departamento a su cargo. Por último, se calcula el salario de cada empleado según las reglas de su categoría y se suma a la nómina, que al final resulta en $128,500.

Conclusiones

La práctica me permitio entender como usarla herencia evita repetir código: lo común se escribe una sola vez en la superclase y cada subclase solo define lo que la hace distinta. También se observó que una variable de la superclase puede guardar objetos de sus subclases, y que al llamar a un método sobrescrito Java ejecuta la versión del objeto real. Esto nos puede ahorrar demasiado tiempo a la hora de programar proyectos mas grandes.

Pruebas y Resultados:

```
Gerente: Laura Méndez (No. G-001)
  Hola, soy Laura Méndez y trabajo en la
empresa. Dirijo el área de Tecnología.
  -> Responsable del departamento
Tecnología.
  Salario mensual: $57000.0
Desarrollador: Carlos Ruiz (No. D-014)
  Hola, soy Carlos Ruiz y trabajo en la
empresa. Programo en Java.
  Salario mensual: $32500.0
Desarrollador: Ana Torres (No. D-021)
  Hola, soy Ana Torres y trabajo en la
empresa. Programo en Python.
  Salario mensual: $33000.0
Becario: Rafael Alejandro (No. B-102)
  Hola, soy Rafael Alejandro y trabajo en
la empresa. Soy becario de la UNAM.
  Salario mensual: $6000.0
Nómina total: $128500.0
```

Referencias:

- Oracle. (s. f.). *Inheritance (The Java™ Tutorials)*. Oracle. https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html
- Oracle. (s. f.). *Java documentation*. Oracle. https://docs.oracle.com/en/java/
- Deitel, P., & Deitel, H. (2017). *Java: How to program* (11th ed.). Pearson.